Your Starter Pack to Agentic Automation [Tamil Edition]

Autonomous Agent — Use Case & Problem Statement

AI HR Policy Assistant | Human Resources Domain

Detail	Description
Audience	Engineering & Arts students (challenge exercise)
Agent Type	Autonomous Agent (no human-in-the-loop)
Platform	UiPath Autopilot / Agent Builder
Difficulty	Moderate — mirrors reference career-advisor pattern

1. Problem Statement

Employees frequently email HR with questions about policy specifics -- how many casual leave days remain, the maximum travel allowance for a given city tier, or the notice period for resignation. Today, HR executives manually search through PDF policy handbooks and spreadsheets to answer each query, which is slow, inconsistent across responders, and creates a bottleneck during appraisal or year-end periods when query volume spikes.

Design and build an Autonomous UiPath Agent, the AI HR Policy Assistant, that reads an incoming employee query about HR policy, retrieves and semantically matches it against a Storage Bucket-backed Index of HR policy documents, and independently returns a clear, cited policy answer by email -- without any human approval step -- using an Index (RAG context) for retrieval and a Send Email tool for the final action.

2. System Prompt

```
You are the AI HR Policy Assistant, designed to help employees understand
HR policies such as leave entitlements, travel allowance limits, and
reimbursement rules, and to send email communication using the Send Email
tool. The inputs (email subject and body) are passed from the agent result
JSON at runtime.

# Process Overview
1. Receive and parse an employee email body containing a policy question
2. Extract the relevant policy topic (e.g., leave type, allowance category,
   grade/band, city tier)
3. Search the 'HR_Policy_FOUND_IDX' context for the matching policy clause
4. Compose an email answer based on the search result
5. Return a JSON response including the email details
6. Use the Send Email tool to send the email irrespective of match found or
   not
# Input Parsing
When receiving an email body (e.g., "Hi HR, could you tell me how many
casual leave days I have left and what the travel allowance limit is for
a Tier-2 city? Thanks, Priya"), follow these steps:
1. Ignore salutations, greetings, and sign-offs
2. Identify each distinct policy question in the email (there may be more
   than one)
```

```
# Search Process
1. Use the 'HR_Policy_FOUND_IDX' context (provided via RAG retrieval)
2. Perform semantic matching ONLY on the following columns:
   - 'Policy Topic'
   - 'Employee Grade / Band'
   - 'Clause Description'
3. Use exact matching for other fields if provided (e.g., 'Policy Code',
   'Effective Date')
4. Perform semantic matching (meaning-aware, tolerant to synonyms/
   paraphrases) only on the specified columns
5. Use Asia/Kolkata timezone for any relative date interpretation

Rules:
- Do not invent or hallucinate index data
- Return only the highest-confidence semantic match per question
- If multiple matches are found, return the one with the highest
  similarity across all fields
- If a question has no matching policy clause, say so plainly rather than
  guessing
```

3. User Prompt

```
The employee will provide an email body containing one or more HR policy
questions. Here is their message:
{{PolicyQueryEmailBody}}

Please extract each policy question from this email body, ignoring any
irrelevant parts such as greetings or sign-offs. Then proceed with the
search and response process as outlined in the system prompt. After
generating the JSON response, use the Send Email tool on either case --
match found or not found:
1. Pass the "email_subject" from the JSON response to the EmailSubject
   parameter
2. Pass the "email_body" from the JSON response to the EmailBody
   parameter
Keep HTML formatting in the Agent result while passing the email body.
```

4. Input Schema

```
{
  "PolicyQueryEmailBody": "string"
}
```

5. Output Schema

```
{
  "status": "MatchFound" | "PartialMatch" | "NoMatchFound" | "Error",
  "message": "string",
  "matched_clauses": [
    {
```

```
      "policy_topic": "string",
      "clause_summary": "string"
    }
  ],
  "email_subject": "string",
  "email_body": "string"
}
```

6. Storage Bucket & Index Setup

Configure the following Orchestrator assets before publishing the agent:

Component	Details
Storage Bucket	HR_Policy_SB -- stores HR policy handbooks, leave charts, and allowance tables
Index Name	HR_Policy_FOUND_IDX
Data Source	Storage Bucket (HR_Policy_SB)
Description	Used to retrieve HR policy clauses on leave, travel allowance, and related entitlements
Ingestion Status	Must show "Successful" before agent testing

7. Tools & Context

Tools

Tool	Purpose
Send Email (SendEmail)	Sends an email based on the agent-generated email subject and body

Contexts

Context	Purpose
HR_Policy_FOUND_IDX	Used to get information on HR policy clauses (leave, travel allowance, reimbursement, etc.)

8. Response Formatting Templates

Match Found

```
{
  "status": "MatchFound",
  "message": "The relevant policy clause(s) have been found:",
  "matched_clauses": [
    {
      "policy_topic": "[Topic from the matched record]",
      "clause_summary": "[Clause text from the matched record]"
    }
  ]
}
```

No Match Found

```
{
  "status": "NoMatchFound",
  "message": "No matching policy clause found in HR_Policy_FOUND_IDX"
}
```

Error Accessing Index

```
{
  "status": "Error",
  "message": "Unable to access HR_Policy_FOUND_IDX. Please try again
  later."
}
```

9. Email Composition Templates

For Match Found

```
Subject: Your HR Policy Query -- Answer Enclosed

Dear Employee,
Thank you for reaching out. Here is the information you requested:
[Policy Topic]: [Clause Summary]

If you have further questions, feel free to write back to HR.

Best regards, HR Policy Team
```

For No Match Found

```
Subject: Your HR Policy Query -- Follow-up Needed

Dear Employee,
We could not find a matching policy clause for your query in our
current records. Our HR team will review this and get back to you
shortly.

Best regards, HR Policy Team
```

10. Expected Learning Outcome

- Configure a Storage Bucket and an Index for RAG-based context retrieval in UiPath
- Design a system prompt that performs semantic matching over specific indexed policy columns
- Handle multi-question emails and return structured, per-question matches
- Define structured JSON input/output schemas for an autonomous HR retrieval agent
- Publish and test a fully autonomous, index-driven agent in UiPath Agent Builder

Note: This use case is intentionally fully autonomous -- the agent performs retrieval, matching, email composition, and sending without an escalation or human-review step, demonstrating end-to-end autonomous decision-making for the workshop challenge.